

DS 669

Reinforcement Learning

Programming Assignment #0

In this assignment, you will get familiar with `FrozenLake-v1`, an environment from Gymnasium library, which will be applied in Assignment #1.

Submission:

- You need to finish the assignment by finishing all parts in each section. If the task is marked as **Question**, please answer it in your words clearly. If the task asks for a screenshot, the screenshot must cover all the answers.
 - Make sure you number each of your responses so it is easy to match responses with exercises.
 - When finished, submit your report in **PDF** form, along with the `code_base.py` and `get_args.py` files on Canvas. **Please DO NOT** zip all your files together.
-

Setup: You need Python3 (versions ≥ 3.8) to complete this assignment. You also need to install the libraries required by running:

- **`pip install gymnasium==0.29.1`**
- **`pip install numpy==1.26.3`**

Please use **`pip3`** instead of **`pip`** to install the libraries if you also have Python 2.x on your computer.

I Environment Instruction

FrozenLake-v1 is a simple gridworld environment where the agent navigate a frozen lake to reach a goal while avoiding holes that would make it fall through the ice. You can find the tutorial for FrozenLake-v1 [here](#), and the code for FrozenLake-v1 [here](#). Please complete this assignment using the provided materials from these two sources.

- Overview
 - Environment Name: FrozenLake-v1
 - Objective: The agent starts at a fixed position on the grid and must navigate to a goal position while avoiding holes.
 - Grid Layout: The grid is typically 4x4 or 8x8 squares. Figure 1 is the screenshot for a 4x4 map.



Figure 1: Frozen Lake 4x4 map

- Grid Element
 - S (Start): The starting point of the agent.
 - F (Frozen): Safe frozen lake tiles where the agent can walk.
 - H (Hole): Dangerous holes in the lake; if the agent steps on one, it falls in and the episode ends with a negative reward.
 - G (Goal): The goal tile; reaching this tile ends the episode with a positive reward.
- Rewards
 - Reaching goal: +1 reward.
 - Falling into hole: 0 reward.
 - Reach frozen: 0 reward.

- Observation Space
 - The observation is a value representing the agent's current position as $current_row * nrow + current_col$ (where both the row and col start at 0).
- Action Space
 - 0: Move left
 - 1: Move down
 - 2: Move right
 - 3: Move up
- Episode and Episode End
 - Episode:
 - * An episode is defined as the sequence of interactions from the initial state to a terminal state.
 - Episode ends if the following happens:
 - * Termination:
 - The agent moves into a hole
 - The agent reaches the goal at $max(nrow) * max(ncol) - 1$ (location[$max(nrow) - 1$, $max(ncol) - 1$])
 - * Truncation (when using the time_limit wrapper):
 - The length of the episode is 100 for 4x4 environment, 200 for 8x8 environment.
- Dynamics
 - Stochastic Environment: By default, the environment is slippery, meaning that there is a chance the agent will slide to an adjacent tile rather than moving directly to the intended tile. This introduces uncertainty and requires the agent to learn a policy that accounts for this stochastic behaviour.

II Environment Initialization (33%)

`gym.make` is the function to initialize the environment, when you call `gym.make`:

- The input of `gym.make`:
 - environment name (string): 'FrozenLake-v1' used in this assignment.
 - desc (list of string): used to specify maps non-preloaded maps, None used in this assignment.
 - map_name (string): used to specify the size of map, '4x4' or '8x8' used in this assignment.
 - render_mode (string): used to specify how the current state of the environment is displayed or output, 'human' used in this assignment to be helpful for visually inspecting how the agent is interacting with the environment step-by-step.

- `is_slippery` (bool): used to specify if the environment is stochastic, `False` used in this assignment.
- The output of `gym.make`:
 - environment object
- (a) Based on the above instruction, please compile `code_base.py` file without changing any default code or settings.

Question: What did you get? Please take a screenshot of your result.
- (b) Can you initialize a 8x8 size Frozen Lake environment? Please provide your screenshot of your map.

(Hint: The only difference between a 4x4 size environment and a 8x8 size environment is the `map_name` parameter in `gym.make`.)

III Environment Functions (33%)

There are two functions in FrozenLake environment, `env.step` and `env.reset`, that will be frequently used in subsequent assignment. Here is a brief instruction of these two functions.

When you call the `env.step`:

- The input of `env.step`:
 - action (ActType): an action the agent provides to update the environment state.
- The returns of `env.step`:
 - observation (ObsType): the next observation after taking actions.
 - reward (SupportsFloat): The reward as a result of taking the action.
 - done (bool): Whether the agent reaches the terminal state (as defined under the MDP of the task) which can be positive or negative. If true, the user needs to call the reset.
 - `False`: not used in this assignment.
 - probability (dict): transition probability for the state

When you call the `env.reset`:

- The input of `env.reset`:
 - `None`
- The returns of `env.reset`:
 - observation (ObsType): the initial state.
 - probability (dict): transition probability for the state set to be 1.
- (a) Please change '`-mode`' to '`test_action`' in `get_args.py` file and then compile `code_base.py` file.

Question: What is the initial state? Where is its location on the map?

(Hint: The location is represented as `[#row, #col]`, e.g., the bottom-left corner is `[3,0]`.)
- (b) **Question:** At initial state, with action 1, what is next state? Where is its location on the map?

IV Environment Test (34%)

In this part, you will help the agent move starting from the initial point in a **8x8** Frozen Lake environment. To begin, please set '-mode' to be 'test_moves' in `get_args.py` file.

- (a) Without any changing any default settings, please compile the `code_base.py` file, you will see the agent moving from the initial point.

Question: Did the agent reach the terminal? Did the episode end?

(**Hint:** The definition of episode and episode end is declared in part I.)

- (b) Can you help the agent move from starting position to the terminal position in the shortest path? Please modify '-actions's default value in the `get_args.py` file, and then compile the `code_base.py` file.

Question: How many steps does the agent need, at minimum, to reach the terminal point from the starting point? Please take a screenshot of your result.